FILE: biz_bp/app.py

```python
"""LocalLens Flask application entry point."""

from __future__ import annotations

import os

from flask import Flask, jsonify
from flask_cors import CORS
from werkzeug.exceptions import HTTPException

from blueprints.auth.auth import auth_bp
from blueprints.businesses.businesses import businesses_bp
from blueprints.reviews.reviews import reviews_bp
from globals import DB_NAME, ensure_indexes


def create_app() -> Flask:
    app = Flask(__name__)
    app.config["JSON_SORT_KEYS"] = False
    CORS(app, resources={r"/api/*": {"origins": "*"}})

    app.register_blueprint(auth_bp)
    app.register_blueprint(businesses_bp)
    app.register_blueprint(reviews_bp)

    @app.get("/health")
    def health():
        return jsonify({"status": "ok", "database": DB_NAME})

    @app.cli.command("init-db")
    def init_db_command():
        """Create MongoDB indexes used by the API."""
        ensure_indexes()
        print("MongoDB indexes created.")

    @app.errorhandler(HTTPException)
    def handle_http_exception(error: HTTPException):
        return jsonify({"error": error.description}), error.code

    @app.errorhandler(Exception)
    def handle_unexpected_exception(error: Exception):
        app.logger.exception("Unhandled application error")
        return jsonify({"error": "Internal server error"}), 500

    return app


app = create_app()


if __name__ == "__main__":
    app.run(
        host=os.getenv("HOST", "127.0.0.1"),
        port=int(os.getenv("PORT", "5000")),
        debug=os.getenv("FLASK_DEBUG", "0") == "1",
    )
```

FILE: biz_bp/globals.py

```python
"""Shared configuration and MongoDB collections for the LocalLens API."""

from __future__ import annotations

import os
import secrets
from pathlib import Path

from pymongo import ASCENDING, DESCENDING, GEOSPHERE, TEXT, MongoClient


def _load_local_env() -> None:
    """Load a gitignored .env.local without adding a dotenv dependency."""
    env_path = Path(__file__).with_name(".env.local")
    if not env_path.exists():
        return
    for raw_line in env_path.read_text(encoding="utf-8").splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
```

```
        key, value = line.split("=", 1)
        os.environ.setdefault(key.strip(), value.strip())


_load_local_env()

MONGO_URI = os.getenv("MONGO_URI", "mongodb://127.0.0.1:27017")
DB_NAME = os.getenv("MONGO_DB_NAME", "locallens")

# A generated development secret is safe to keep in memory, but it changes on
# restart. Production/coursework runs should set JWT_SECRET explicitly.
JWT_SECRET = os.getenv("JWT_SECRET") or secrets.token_hex(32)
JWT_ALGORITHM = "HS256"
JWT_EXPIRY_SECONDS = int(os.getenv("JWT_EXPIRY_SECONDS", "28800"))

CLIENT = MongoClient(MONGO_URI, serverSelectionTimeoutMS=3000)
DB = CLIENT[DB_NAME]

users_collection = DB["users"]
businesses_collection = DB["businesses"]
reviews_collection = DB["reviews"]
blacklist_collection = DB["blacklist"]


def ensure_indexes() -> None:
    """Create indexes required by authentication and advanced queries."""
    users_collection.create_index([("username", ASCENDING)], unique=True)
    blacklist_collection.create_index([("jti", ASCENDING)], unique=True)
    blacklist_collection.create_index(
        [("expires_at", ASCENDING)], expireAfterSeconds=0
    )
    businesses_collection.create_index([("location", GEOSPHERE)])
    businesses_collection.create_index(
        [("name", TEXT), ("description", TEXT), ("town", TEXT)]
    )
    businesses_collection.create_index(
        [("town", ASCENDING), ("categories", ASCENDING)]
    )
    reviews_collection.create_index(
        [("business_id", ASCENDING), ("created_at", DESCENDING)]
    )
    reviews_collection.create_index([("user_id", ASCENDING)])
```

FILE: biz_bp/decorators.py

```
"""JWT and role-based access-control decorators."""

from __future__ import annotations

from functools import wraps
from typing import Any, Callable

import jwt
from bson import ObjectId
from flask import g, jsonify, request

from globals import (
    JWT_ALGORITHM,
    JWT_SECRET,
    blacklist_collection,
    users_collection,
)


def _error(message: str, status: int):
    return jsonify({"error": message}), status


def jwt_required(view: Callable[..., Any]) -> Callable[..., Any]:
    """Validate a Bearer JWT, reject logout-blacklisted tokens, and load the user."""

    @wraps(view)
    def wrapper(*args, **kwargs):
        auth_header = request.headers.get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            return _error("Authorization header must use Bearer <token>", 401)

        token = auth_header[7:].strip()
        if not token:
            return _error("Bearer token is missing", 401)

        try:
```

```
            payload = jwt.decode(
                token,
                JWT_SECRET,
                algorithms=[JWT_ALGORITHM],
            )
        except jwt.ExpiredSignatureError:
            return _error("Token has expired", 401)
        except jwt.InvalidTokenError:
            return _error("Token is invalid", 401)

        if not payload.get("sub") or not payload.get("jti") or not payload.get("exp"):
            return _error("Token is missing required claims", 401)

        if blacklist_collection.find_one({"jti": payload["jti"]}):
            return _error("Token has been revoked", 401)

        if not ObjectId.is_valid(payload["sub"]):
            return _error("Token subject is invalid", 401)

        user = users_collection.find_one({"_id": ObjectId(payload["sub"])})
        if not user:
            return _error("Token user no longer exists", 401)

        g.current_user = user
        g.jwt_payload = payload
        g.jwt_token = token
        return view(*args, **kwargs)

    return wrapper


def admin_required(view: Callable[..., Any]) -> Callable[..., Any]:
    """Require the authenticated user loaded by jwt_required to be an admin."""

    @wraps(view)
    def wrapper(*args, **kwargs):
        user = getattr(g, "current_user", None)
        if not user or not user.get("admin", False):
            return _error("Administrator permission required", 403)
        return view(*args, **kwargs)

    return wrapper
```

```
================================================================================================
FILE: biz_bp/utils.py
================================================================================================
"""Small shared validation and JSON serialisation helpers."""

from __future__ import annotations

from datetime import date, datetime, timezone
from typing import Any

from bson import ObjectId
from flask import request


def utcnow() -> datetime:
    """Return a timezone-aware UTC timestamp."""
    return datetime.now(timezone.utc)


def json_safe(value: Any) -> Any:
    """Convert BSON/Python values into JSON-compatible values."""
    if isinstance(value, ObjectId):
        return str(value)
    if isinstance(value, (datetime, date)):
        return value.isoformat()
    if isinstance(value, dict):
        return {str(key): json_safe(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [json_safe(item) for item in value]
    return value


def parse_object_id(value: str, field_name: str = "id") -> ObjectId:
    """Parse a MongoDB ObjectId or raise ValueError."""
    if not ObjectId.is_valid(value):
        raise ValueError(f"{field_name} must be a 24-character MongoDB ObjectId")
    return ObjectId(value)


def parse_pagination() -> tuple[int, int, int]:
```

```
    """Read pn/ps query parameters and return page, size and skip."""
    try:
        page = int(request.args.get("pn", "1"))
        page_size = int(request.args.get("ps", "20"))
    except ValueError as exc:
        raise ValueError("pn and ps must be integers") from exc

    if page < 1:
        raise ValueError("pn must be greater than or equal to 1")
    if page_size < 1 or page_size > 100:
        raise ValueError("ps must be between 1 and 100")
    return page, page_size, (page - 1) * page_size


def pagination_payload(page: int, page_size: int, total: int) -> dict[str, Any]:
    """Build a consistent pagination metadata object."""
    pages = (total + page_size - 1) // page_size if total else 0
    return {
        "page": page,
        "page_size": page_size,
        "total": total,
        "pages": pages,
    }
```

```
================================================================================================
FILE: biz_bp/blueprints/__init__.py
================================================================================================
"""Flask blueprints for the LocalLens API."""
```

```
================================================================================================
FILE: biz_bp/blueprints/auth/auth.py
================================================================================================
"""Registration, JWT login and logout endpoints."""

from __future__ import annotations

import re
from datetime import timedelta, timezone
from datetime import datetime
from uuid import uuid4

import bcrypt
import jwt
from bson import ObjectId
from flask import Blueprint, g, jsonify, request
from pymongo.errors import DuplicateKeyError

from decorators import jwt_required
from globals import (
    JWT_ALGORITHM,
    JWT_EXPIRY_SECONDS,
    JWT_SECRET,
    blacklist_collection,
    users_collection,
)
from utils import json_safe, utcnow

auth_bp = Blueprint("auth", __name__, url_prefix="/api/v1.0")

USERNAME_PATTERN = re.compile(r"^[A-Za-z0-9_-]{3,50}$")
EMAIL_PATTERN = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")


def _serialise_user(user: dict) -> dict:
    return {
        "id": str(user["_id"]),
        "username": user["username"],
        "email": user.get("email"),
        "admin": bool(user.get("admin", False)),
        "created_at": json_safe(user.get("created_at")),
    }


def _credentials_from_request() -> tuple[object, object]:
    payload = request.get_json(silent=True) or {}
    username = payload.get("username", request.args.get("username"))
    password = payload.get("password", request.args.get("password"))
    return username, password


def _issue_token(user: dict) -> dict:
    issued_at = utcnow()
    expires_at = issued_at + timedelta(seconds=JWT_EXPIRY_SECONDS)
```

```
    payload = {
        "sub": str(user["_id"]),
        "username": user["username"],
        "admin": bool(user.get("admin", False)),
        "iat": issued_at,
        "exp": expires_at,
        "jti": uuid4().hex,
    }
    token = jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)
    return {
        "token": token,
        "token_type": "Bearer",
        "expires_in": JWT_EXPIRY_SECONDS,
        "user": _serialise_user(user),
    }


@auth_bp.post("/users")
def register_user():
    """Register a non-admin user."""
    payload = request.get_json(silent=True) or {}
    username = payload.get("username")
    password = payload.get("password")
    email = payload.get("email")

    if not isinstance(username, str) or not USERNAME_PATTERN.fullmatch(username):
        return jsonify(
            {"error": "username must be 3-50 letters, numbers, '_' or '-'"}
        ), 400
    if not isinstance(password, str) or len(password) < 8:
        return jsonify({"error": "password must contain at least 8 characters"}), 400
    if email is not None and (
        not isinstance(email, str) or not EMAIL_PATTERN.fullmatch(email)
    ):
        return jsonify({"error": "email is invalid"}), 400

    password_hash = bcrypt.hashpw(
        password.encode("utf-8"), bcrypt.gensalt()
    ).decode("utf-8")
    document = {
        "username": username,
        "email": email,
        "password_hash": password_hash,
        "admin": False,
        "created_at": utcnow(),
    }

    try:
        result = users_collection.insert_one(document)
    except DuplicateKeyError:
        return jsonify({"error": "username already exists"}), 409

    document["_id"] = result.inserted_id
    return jsonify({"user": _serialise_user(document)}), 201


@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    """Authenticate with username/password and return a JWT.

    POST JSON is the preferred form. GET query parameters are retained because
    the module endpoint specification explicitly lists GET /login.
    """
    username, password = _credentials_from_request()
    if not isinstance(username, str) or not isinstance(password, str):
        return jsonify({"error": "username and password are required"}), 400

    user = users_collection.find_one({"username": username})
    if not user:
        return jsonify({"error": "Invalid username or password"}), 401

    stored_hash = user.get("password_hash", "")
    if isinstance(stored_hash, bytes):
        stored_hash_bytes = stored_hash
    elif isinstance(stored_hash, str):
        stored_hash_bytes = stored_hash.encode("utf-8")
    else:
        stored_hash_bytes = b""

    try:
        password_matches = bcrypt.checkpw(
            password.encode("utf-8"), stored_hash_bytes
        )
```

```
    except ValueError:
        password_matches = False

    if not password_matches:
        return jsonify({"error": "Invalid username or password"}), 401

    return jsonify(_issue_token(user)), 200


@auth_bp.get("/logout")
@jwt_required
def logout():
    """Revoke the current JWT by adding its jti to the blacklist."""
    payload = g.jwt_payload
    expires_at = datetime.fromtimestamp(int(payload["exp"]), tz=timezone.utc)
    blacklist_collection.update_one(
        {"jti": payload["jti"]},
        {
            "$setOnInsert": {
                "jti": payload["jti"],
                "user_id": ObjectId(payload["sub"]),
                "expires_at": expires_at,
                "created_at": utcnow(),
            }
        },
        upsert=True,
    )
    return jsonify({"message": "Logged out successfully"}), 200
```

==========

FILE: biz_bp/blueprints/businesses/businesses.py

==========

```
"""Business CRUD, aggregation and geospatial query endpoints."""

from __future__ import annotations

import re
from typing import Any

from flask import Blueprint, jsonify, request
from pymongo import ASCENDING, DESCENDING

from decorators import admin_required, jwt_required
from globals import businesses_collection, reviews_collection
from utils import (
    json_safe,
    pagination_payload,
    parse_object_id,
    parse_pagination,
    utcnow,
)

businesses_bp = Blueprint(
    "businesses", __name__, url_prefix="/api/v1.0/businesses"
)

ALLOWED_BUSINESS_FIELDS = {
    "name",
    "town",
    "address",
    "postcode",
    "categories",
    "description",
    "location",
    "price_level",
    "active",
}
SORT_FIELDS = {"name", "town", "rating", "review_count", "created_at"}


def _validate_business_payload(
    payload: dict[str, Any], *, partial: bool = False
) -> tuple[dict[str, Any], list[str]]:
    """Validate and normalise a business request body."""
    errors: list[str] = []
    clean: dict[str, Any] = {}
    unknown = sorted(set(payload) - ALLOWED_BUSINESS_FIELDS)
    if unknown:
        errors.append(f"Unsupported fields: {', '.join(unknown)}")

    required = {"name", "town", "location"}
    if not partial:
        missing = sorted(field for field in required if field not in payload)
```

```
        if missing:
            errors.append(f"Missing required fields: {', '.join(missing)}")

    if "name" in payload:
        name = payload["name"]
        if not isinstance(name, str) or not name.strip() or len(name.strip()) > 200:
            errors.append("name must be a non-empty string of at most 200 characters")
        else:
            clean["name"] = name.strip()

    for field in ("town", "address", "postcode"):
        if field in payload:
            value = payload[field]
            if not isinstance(value, str) or not value.strip():
                errors.append(f"{field} must be a non-empty string")
            else:
                clean[field] = value.strip()

    if "description" in payload:
        description = payload["description"]
        if not isinstance(description, str) or len(description) > 5000:
            errors.append("description must be a string of at most 5000 characters")
        else:
            clean["description"] = description

    if "categories" in payload:
        categories = payload["categories"]
        if (
            not isinstance(categories, list)
            or not categories
            or len(categories) > 10
            or any(
                not isinstance(item, str) or not item.strip() or len(item.strip()) > 80
                for item in categories
            )
        ):
            errors.append("categories must be a list of 1-10 non-empty strings")
        else:
            clean["categories"] = list(
                dict.fromkeys(item.strip() for item in categories)
            )

    if "location" in payload:
        location = payload["location"]
        valid_location = (
            isinstance(location, dict)
            and location.get("type") == "Point"
            and isinstance(location.get("coordinates"), list)
            and len(location["coordinates"]) == 2
            and all(
                isinstance(value, (int, float)) and not isinstance(value, bool)
                for value in location["coordinates"]
            )
        )
        if not valid_location:
            errors.append(
                "location must be GeoJSON {'type':'Point','coordinates':[lon,lat]}"
            )
        else:
            longitude, latitude = location["coordinates"]
            if not (-180 <= longitude <= 180 and -90 <= latitude <= 90):
                errors.append("location coordinates are outside valid longitude/latitude")
            else:
                clean["location"] = {
                    "type": "Point",
                    "coordinates": [float(longitude), float(latitude)],
                }

    if "price_level" in payload:
        price_level = payload["price_level"]
        if (
            not isinstance(price_level, int)
            or isinstance(price_level, bool)
            or price_level not in {1, 2, 3, 4}
        ):
            errors.append("price_level must be an integer from 1 to 4")
        else:
            clean["price_level"] = price_level

    if "active" in payload:
        if not isinstance(payload["active"], bool):
            errors.append("active must be a boolean")
        else:
```

```
            clean["active"] = payload["active"]

    return clean, errors


def _sort_spec() -> list[tuple[str, int]]:
    requested = request.args.get("sort", "-created_at")
    descending = requested.startswith("-")
    field = requested[1:] if descending else requested
    if field not in SORT_FIELDS:
        raise ValueError(f"sort must be one of: {', '.join(sorted(SORT_FIELDS))}")
    direction = DESCENDING if descending else ASCENDING
    return [(field, direction), ("_id", ASCENDING)]


@businesses_bp.get("")
def list_businesses():
    """List businesses with pagination, text/multi-value filters and sorting."""
    try:
        page, page_size, skip = parse_pagination()
        sort_spec = _sort_spec()
        query = request.args.get("q", "").strip()
        town = request.args.get("town", "").strip()
        categories = [
            value.strip() for value in request.args.getlist("category") if value.strip()
        ]
        min_rating = request.args.get("min_rating")
        max_rating = request.args.get("max_rating")
        active = request.args.get("active")
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    filters: dict[str, Any] = {}
    if query:
        escaped = re.escape(query)
        filters["$or"] = [
            {"name": {"$regex": escaped, "$options": "i"}},
            {"description": {"$regex": escaped, "$options": "i"}},
            {"categories": {"$regex": escaped, "$options": "i"}},
        ]
    if town:
        filters["town"] = {"$regex": f"^{re.escape(town)}$", "$options": "i"}
    if categories:
        filters["categories"] = {"$all": categories}

    if min_rating is not None or max_rating is not None:
        rating_filter: dict[str, float] = {}
        try:
            if min_rating is not None:
                rating_filter["$gte"] = float(min_rating)
            if max_rating is not None:
                rating_filter["$lte"] = float(max_rating)
        except ValueError:
            return jsonify({"error": "min_rating and max_rating must be numbers"}), 400
        if any(not 0 <= value <= 5 for value in rating_filter.values()):
            return jsonify({"error": "rating filters must be between 0 and 5"}), 400
        filters["rating"] = rating_filter

    if active is not None:
        if active.lower() not in {"true", "false"}:
            return jsonify({"error": "active must be true or false"}), 400
        filters["active"] = active.lower() == "true"

    total = businesses_collection.count_documents(filters)
    documents = (
        businesses_collection.find(filters)
        .sort(sort_spec)
        .skip(skip)
        .limit(page_size)
    )
    return jsonify(
        {
            "items": json_safe(list(documents)),
            **pagination_payload(page, page_size, total),
        }
    ), 200


@businesses_bp.post("")
@jwt_required
def create_business():
    """Create a business as an authenticated user."""
    payload = request.get_json(silent=True)
```

```
    if not isinstance(payload, dict):
        return jsonify({"error": "JSON object body is required"}), 400

    clean, errors = _validate_business_payload(payload)
    if errors:
        return jsonify({"error": "Validation failed", "details": errors}), 400

    duplicate = businesses_collection.find_one(
        {
            "name": {"$regex": f"^{re.escape(clean['name'])}$", "$options": "i"},
            "town": {"$regex": f"^{re.escape(clean['town'])}$", "$options": "i"},
        }
    )
    if duplicate:
        return jsonify({"error": "A business with this name and town exists"}), 409

    now = utcnow()
    document = {
        **clean,
        "rating": 0.0,
        "review_count": 0,
        "created_at": now,
        "updated_at": now,
    }
    result = businesses_collection.insert_one(document)
    document["_id"] = result.inserted_id
    return (
        jsonify(json_safe(document)),
        201,
        {"Location": f"/api/v1.0/businesses/{result.inserted_id}"},
    )


@businesses_bp.get("/advanced")
def advanced_business_search():
    """Run the required multi-stage aggregation pipeline."""
    try:
        page, page_size, skip = parse_pagination()
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    category = request.args.get("category", "").strip()
    min_rating_raw = request.args.get("min_rating", "0")
    try:
        min_rating = float(min_rating_raw)
    except ValueError:
        return jsonify({"error": "min_rating must be a number"}), 400
    if not 0 <= min_rating <= 5:
        return jsonify({"error": "min_rating must be between 0 and 5"}), 400

    post_unwind_match: dict[str, Any] = {"categories": {"$exists": True}}
    if category:
        post_unwind_match["categories"] = category

    pipeline = [
        {"$match": {"rating": {"$gte": min_rating}, "active": True}},
        {
            "$project": {
                "name": 1,
                "town": 1,
                "address": 1,
                "rating": 1,
                "review_count": 1,
                "categories": 1,
            }
        },
        {"$unwind": "$categories"},
        {"$match": post_unwind_match},
        {"$sort": {"rating": -1, "_id": 1}},
        {"$skip": skip},
        {"$limit": page_size},
    ]
    documents = list(businesses_collection.aggregate(pipeline))
    return jsonify(
        {
            "items": json_safe(documents),
            "page": page,
            "page_size": page_size,
            "pipeline_stages": [
                "$match",
                "$project",
                "$unwind",
                "$match",
```

```
                "$sort",
                "$skip",
                "$limit",
            ],
        }
    ), 200


@businesses_bp.get("/stats/by-town")
def business_stats_by_town():
    """Group businesses by town and return counts plus average ratings."""
    limit_raw = request.args.get("limit", "50")
    try:
        limit = int(limit_raw)
    except ValueError:
        return jsonify({"error": "limit must be an integer"}), 400
    if not 1 <= limit <= 200:
        return jsonify({"error": "limit must be between 1 and 200"}), 400

    pipeline = [
        {"$match": {"town": {"$exists": True, "$ne": ""}}},
        {
            "$group": {
                "_id": "$town",
                "business_count": {"$sum": 1},
                "average_rating": {"$avg": "$rating"},
                "review_count": {"$sum": "$review_count"},
            }
        },
        {"$sort": {"business_count": -1, "_id": 1}},
        {"$limit": limit},
        {
            "$project": {
                "_id": 0,
                "town": "$_id",
                "business_count": 1,
                "average_rating": {"$round": ["$average_rating", 2]},
                "review_count": 1,
            }
        },
    ]
    return jsonify({"items": json_safe(list(businesses_collection.aggregate(pipeline)))}), 200


@businesses_bp.get("/nearby")
def nearby_businesses():
    """Find businesses near a longitude/latitude using $geoNear."""
    longitude_raw = request.args.get("longitude")
    latitude_raw = request.args.get("latitude")
    try:
        longitude = float(longitude_raw)
        latitude = float(latitude_raw)
    except (TypeError, ValueError):
        return jsonify({"error": "longitude and latitude are required numbers"}), 400
    if not (-180 <= longitude <= 180 and -90 <= latitude <= 90):
        return jsonify({"error": "coordinates are outside valid ranges"}), 400

    try:
        page, page_size, skip = parse_pagination()
        max_distance = float(request.args.get("max_distance", "5000"))
        min_distance = float(request.args.get("min_distance", "0"))
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400
    if max_distance <= 0 or min_distance < 0 or min_distance >= max_distance:
        return jsonify(
            {"error": "distances require 0 <= min_distance < max_distance"}
        ), 400

    pipeline = [
        {
            "$geoNear": {
                "near": {"type": "Point", "coordinates": [longitude, latitude]},
                "distanceField": "distance_m",
                "maxDistance": max_distance,
                "minDistance": min_distance,
                "spherical": True,
                "query": {"active": True},
            }
        },
        {"$skip": skip},
        {"$limit": page_size},
        {
            "$project": {
```

```
                "name": 1,
                "town": 1,
                "address": 1,
                "categories": 1,
                "rating": 1,
                "review_count": 1,
                "location": 1,
                "distance_m": {"$round": ["$distance_m", 2]},
            }
        },
    ]
    return jsonify(
        {
            "items": json_safe(list(businesses_collection.aggregate(pipeline))),
            "page": page,
            "page_size": page_size,
        }
    ), 200


@businesses_bp.get("/<string:business_id>")
def get_business(business_id: str):
    """Return one business by MongoDB ObjectId."""
    try:
        object_id = parse_object_id(business_id, "business_id")
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    document = businesses_collection.find_one({"_id": object_id})
    if not document:
        return jsonify({"error": "Business not found"}), 404
    return jsonify(json_safe(document)), 200


@businesses_bp.put("/<string:business_id>")
@jwt_required
def update_business(business_id: str):
    """Partially update a business as an authenticated user."""
    try:
        object_id = parse_object_id(business_id, "business_id")
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    payload = request.get_json(silent=True)
    if not isinstance(payload, dict):
        return jsonify({"error": "JSON object body is required"}), 400

    clean, errors = _validate_business_payload(payload, partial=True)
    if errors:
        return jsonify({"error": "Validation failed", "details": errors}), 400
    if not clean:
        return jsonify({"error": "At least one supported field is required"}), 400

    clean["updated_at"] = utcnow()
    result = businesses_collection.update_one({"_id": object_id}, {"$set": clean})
    if result.matched_count == 0:
        return jsonify({"error": "Business not found"}), 404

    document = businesses_collection.find_one({"_id": object_id})
    return jsonify(json_safe(document)), 200


@businesses_bp.delete("/<string:business_id>")
@jwt_required
@admin_required
def delete_business(business_id: str):
    """Delete a business and its reviews as an administrator."""
    try:
        object_id = parse_object_id(business_id, "business_id")
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    result = businesses_collection.delete_one({"_id": object_id})
    if result.deleted_count == 0:
        return jsonify({"error": "Business not found"}), 404
    reviews_collection.delete_many({"business_id": object_id})
    return "", 204
```

```
================================================================================================
FILE: biz_bp/blueprints/reviews/reviews.py
================================================================================================
```

```
"""Review CRUD endpoints nested under a business resource."""

```

```
from __future__ import annotations

from typing import Any

from flask import Blueprint, g, jsonify, request
from pymongo import ASCENDING, DESCENDING

from decorators import admin_required, jwt_required
from globals import businesses_collection, reviews_collection
from utils import (
    json_safe,
    pagination_payload,
    parse_object_id,
    parse_pagination,
    utcnow,
)

reviews_bp = Blueprint("reviews", __name__, url_prefix="/api/v1.0/businesses")

SORT_FIELDS = {"created_at", "updated_at", "rating"}


def _validate_review_payload(
    payload: dict[str, Any], *, partial: bool = False
) -> tuple[dict[str, Any], list[str]]:
    errors: list[str] = []
    clean: dict[str, Any] = {}
    unknown = sorted(set(payload) - {"rating", "text"})
    if unknown:
        errors.append(f"Unsupported fields: {', '.join(unknown)}")

    if not partial:
        missing = sorted(field for field in ("rating", "text") if field not in payload)
        if missing:
            errors.append(f"Missing required fields: {', '.join(missing)}")

    if "rating" in payload:
        rating = payload["rating"]
        if (
            not isinstance(rating, int)
            or isinstance(rating, bool)
            or rating < 1
            or rating > 5
        ):
            errors.append("rating must be an integer from 1 to 5")
        else:
            clean["rating"] = rating

    if "text" in payload:
        text = payload["text"]
        if not isinstance(text, str) or not text.strip() or len(text.strip()) > 5000:
            errors.append("text must be a non-empty string of at most 5000 characters")
        else:
            clean["text"] = text.strip()

    return clean, errors


def _sort_spec() -> list[tuple[str, int]]:
    requested = request.args.get("sort", "-created_at")
    descending = requested.startswith("-")
    field = requested[1:] if descending else requested
    if field not in SORT_FIELDS:
        raise ValueError(f"sort must be one of: {', '.join(sorted(SORT_FIELDS))}")
    return [(field, DESCENDING if descending else ASCENDING), ("_id", ASCENDING)]


def _recalculate_business_rating(business_id) -> None:
    """Recompute denormalised rating and review_count for a business."""
    summary = list(
        reviews_collection.aggregate(
            [
                {"$match": {"business_id": business_id}},
                {
                    "$group": {
                        "_id": "$business_id",
                        "review_count": {"$sum": 1},
                        "rating": {"$avg": "$rating"},
                    }
                },
            ]
        )
    )
```

```
    if summary:
        rating = round(float(summary[0]["rating"]), 2)
        review_count = int(summary[0]["review_count"])
    else:
        rating = 0.0
        review_count = 0
    businesses_collection.update_one(
        {"_id": business_id},
        {"$set": {"rating": rating, "review_count": review_count, "updated_at": utcnow()}},
    )


def _business_or_404(business_id: str):
    business_object_id = parse_object_id(business_id, "business_id")
    business = businesses_collection.find_one({"_id": business_object_id})
    return business_object_id, business


@reviews_bp.get("/<string:business_id>/reviews")
def list_reviews(business_id: str):
    """List reviews for one business with pagination and filters."""
    try:
        business_object_id, business = _business_or_404(business_id)
        page, page_size, skip = parse_pagination()
        sort_spec = _sort_spec()
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    if not business:
        return jsonify({"error": "Business not found"}), 404

    filters: dict[str, Any] = {"business_id": business_object_id}
    rating_raw = request.args.get("rating")
    if rating_raw is not None:
        try:
            rating = int(rating_raw)
        except ValueError:
            return jsonify({"error": "rating must be an integer"}), 400
        if not 1 <= rating <= 5:
            return jsonify({"error": "rating must be between 1 and 5"}), 400
        filters["rating"] = rating

    total = reviews_collection.count_documents(filters)
    documents = (
        reviews_collection.find(filters)
        .sort(sort_spec)
        .skip(skip)
        .limit(page_size)
    )
    return jsonify(
        {
            "business_id": business_id,
            "items": json_safe(list(documents)),
            **pagination_payload(page, page_size, total),
        }
    ), 200


@reviews_bp.post("/<string:business_id>/reviews")
@jwt_required
def create_review(business_id: str):
    """Create one review for a business as an authenticated user."""
    try:
        business_object_id, business = _business_or_404(business_id)
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400
    if not business:
        return jsonify({"error": "Business not found"}), 404

    payload = request.get_json(silent=True)
    if not isinstance(payload, dict):
        return jsonify({"error": "JSON object body is required"}), 400
    clean, errors = _validate_review_payload(payload)
    if errors:
        return jsonify({"error": "Validation failed", "details": errors}), 400

    user_id = g.current_user["_id"]
    duplicate = reviews_collection.find_one(
        {"business_id": business_object_id, "user_id": user_id}
    )
    if duplicate:
        return jsonify({"error": "This user already reviewed this business"}), 409

```

```
    now = utcnow()
    document = {
        "business_id": business_object_id,
        "user_id": user_id,
        "rating": clean["rating"],
        "text": clean["text"],
        "created_at": now,
        "updated_at": now,
    }
    result = reviews_collection.insert_one(document)
    document["_id"] = result.inserted_id
    _recalculate_business_rating(business_object_id)
    return (
        jsonify(json_safe(document)),
        201,
        {
            "Location": (
                f"/api/v1.0/businesses/{business_id}/reviews/{result.inserted_id}"
            )
        },
    )


@reviews_bp.get("/<string:business_id>/reviews/<string:review_id>")
def get_review(business_id: str, review_id: str):
    """Return one review belonging to the specified business."""
    try:
        business_object_id = parse_object_id(business_id, "business_id")
        review_object_id = parse_object_id(review_id, "review_id")
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    if not businesses_collection.find_one({"_id": business_object_id}):
        return jsonify({"error": "Business not found"}), 404

    review = reviews_collection.find_one(
        {"_id": review_object_id, "business_id": business_object_id}
    )
    if not review:
        return jsonify({"error": "Review not found"}), 404
    return jsonify(json_safe(review)), 200


@reviews_bp.put("/<string:business_id>/reviews/<string:review_id>")
@jwt_required
def update_review(business_id: str, review_id: str):
    """Update an owned review; administrators may update any review."""
    try:
        business_object_id = parse_object_id(business_id, "business_id")
        review_object_id = parse_object_id(review_id, "review_id")
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400

    if not businesses_collection.find_one({"_id": business_object_id}):
        return jsonify({"error": "Business not found"}), 404

    review = reviews_collection.find_one(
        {"_id": review_object_id, "business_id": business_object_id}
    )
    if not review:
        return jsonify({"error": "Review not found"}), 404

    current_user = g.current_user
    if review.get("user_id") != current_user["_id"] and not current_user.get(
        "admin", False
    ):
        return jsonify({"error": "You may only edit your own review"}), 403

    payload = request.get_json(silent=True)
    if not isinstance(payload, dict):
        return jsonify({"error": "JSON object body is required"}), 400
    clean, errors = _validate_review_payload(payload, partial=True)
    if errors:
        return jsonify({"error": "Validation failed", "details": errors}), 400
    if not clean:
        return jsonify({"error": "At least one of rating or text is required"}), 400

    clean["updated_at"] = utcnow()
    reviews_collection.update_one({"_id": review_object_id}, {"$set": clean})
    _recalculate_business_rating(business_object_id)
    updated = reviews_collection.find_one({"_id": review_object_id})
    return jsonify(json_safe(updated)), 200

```

```
 252 |
 253 | @reviews_bp.delete("/<string:business_id>/reviews/<string:review_id>")
 254 | @jwt_required
 255 | @admin_required
 256 | def delete_review(business_id: str, review_id: str):
 257 |     """Delete any review as an administrator."""
 258 |     try:
 259 |         business_object_id = parse_object_id(business_id, "business_id")
 260 |         review_object_id = parse_object_id(review_id, "review_id")
 261 |     except ValueError as exc:
 262 |         return jsonify({"error": str(exc)}), 400
 263 |
 264 |     if not businesses_collection.find_one({"_id": business_object_id}):
 265 |         return jsonify({"error": "Business not found"}), 404
 266 |
 267 |     result = reviews_collection.delete_one(
 268 |         {"_id": review_object_id, "business_id": business_object_id}
 269 |     )
 270 |     if result.deleted_count == 0:
 271 |         return jsonify({"error": "Review not found"}), 404
 272 |
 273 |     _recalculate_business_rating(business_object_id)
 274 |     return "", 204
```

```
================================================================================================
FILE: biz_bp/create_users.py
================================================================================================
   1 | """Compatibility entry point required by AGENT.md."""
   2 |
   3 | from scripts.create_users import main
   4 |
   5 |
   6 | if __name__ == "__main__":
   7 |     main()
```

```
================================================================================================
FILE: biz_bp/scripts/import_seed_data.py
================================================================================================
   1 | """Import generated LocalLens JSON with MongoDB’s Extended JSON support."""
   2 |
   3 | from __future__ import annotations
   4 |
   5 | import argparse
   6 | import sys
   7 | from pathlib import Path
   8 |
   9 | from bson import json_util
  10 |
  11 | ROOT = Path(__file__).resolve().parent.parent
  12 | sys.path.insert(0, str(ROOT))
  13 |
  14 | from globals import (  # noqa: E402
  15 |     businesses_collection,
  16 |     ensure_indexes,
  17 |     reviews_collection,
  18 | )
  19 |
  20 |
  21 | def load_documents(path: Path) -> list[dict]:
  22 |     with path.open("r", encoding="utf-8") as handle:
  23 |         return json_util.loads(handle.read())
  24 |
  25 |
  26 | def main() -> None:
  27 |     parser = argparse.ArgumentParser(description="Import LocalLens seed data")
  28 |     parser.add_argument(
  29 |         "--reset",
  30 |         action="store_true",
  31 |         help="Delete existing businesses/reviews before importing (destructive)",
  32 |     )
  33 |     args = parser.parse_args()
  34 |
  35 |     business_path = ROOT / "data" / "businesses.json"
  36 |     review_path = ROOT / "data" / "reviews.json"
  37 |     if not business_path.exists() or not review_path.exists():
  38 |         raise SystemExit("Seed JSON is missing; run scripts/make_seed_data.py first")
  39 |
  40 |     if args.reset:
  41 |         businesses_collection.delete_many({})
  42 |         reviews_collection.delete_many({})
  43 |
  44 |     if businesses_collection.count_documents({}) or reviews_collection.count_documents({}):
  45 |         raise SystemExit(
```

```
            "Target collections are not empty. Re-run with --reset only if deletion is intended."
        )

    businesses = load_documents(business_path)
    reviews = load_documents(review_path)
    businesses_collection.insert_many(businesses)
    reviews_collection.insert_many(reviews)
    ensure_indexes()
    print(f"Imported {len(businesses)} businesses and {len(reviews)} reviews.")


if __name__ == "__main__":
    main()
```

```
================================================================================
FILE: biz_bp/scripts/export_data.py
================================================================================
```

```
"""Export each LocalLens collection as MongoDB Extended JSON.

This is a fallback for environments where the MongoDB Database Tools
(mongoexport) are not installed. It never prints credentials.
"""

from __future__ import annotations

import sys
from pathlib import Path

from bson import json_util

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from globals import DB  # noqa: E402

EXPORT_DIR = ROOT / "data" / "export"
COLLECTIONS = ("businesses", "reviews", "users", "blacklist")


def main() -> None:
    EXPORT_DIR.mkdir(parents=True, exist_ok=True)
    for name in COLLECTIONS:
        documents = list(DB[name].find({}))
        output = EXPORT_DIR / f"{name}.json"
        output.write_text(
            json_util.dumps(documents, indent=2, ensure_ascii=False),
            encoding="utf-8",
        )
        print(f"Exported {len(documents)} documents from {name} to {output}")
    print("Note: users.json contains password hashes, not plaintext passwords.")


if __name__ == "__main__":
    main()
```

```
================================================================================
FILE: biz_bp/scripts/make_seed_data.py
================================================================================
```

```
"""Generate deterministic LocalLens seed JSON for MongoDB import.

The generated data is synthetic and intentionally independent of Yelp. It
contains 120 parent businesses and 600 child reviews, with text, numeric,
date, boolean and GeoJSON values.
"""

from __future__ import annotations

import json
import random
from datetime import datetime, timedelta, timezone
from pathlib import Path

SEED = 661
BUSINESS_COUNT = 120
REVIEWS_PER_BUSINESS = 5
BASE_DATE = datetime(2025, 1, 1, tzinfo=timezone.utc)
ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"

TOWNS = [
    ("Belfast", -5.9301, 54.5973, "BT1"),
    ("Derry", -7.3093, 54.9966, "BT48"),
    ("Lisburn", -6.0580, 54.5162, "BT28"),
```

```
    ("Newry", -6.3374, 54.1751, "BT34"),
    ("Armagh", -6.6540, 54.3503, "BT61"),
    ("Bangor", -5.6689, 54.6534, "BT20"),
    ("Coleraine", -6.6760, 55.1326, "BT52"),
    ("Enniskillen", -7.6380, 54.3438, "BT74"),
]
CATEGORIES = [
    "Cafe",
    "Restaurant",
    "Bakery",
    "Bookshop",
    "Bar",
    "Deli",
    "Gallery",
    "Grocery",
    "Hotel",
    "Museum",
]
NAME_PREFIXES = [
    "Harbour",
    "Linen",
    "Meadow",
    "Oak",
    "Riverside",
    "North",
    "Bright",
    "Old Town",
    "Copper",
    "Willow",
]
NAME_SUFFIXES = [
    "Kitchen",
    "House",
    "Room",
    "Market",
    "Cafe",
    "Studio",
    "Corner",
    "Larder",
    "Social",
    "Garden",
]
DESCRIPTIONS = [
    "Independent local business with seasonal produce.",
    "Neighbourhood venue focused on accessible service.",
    "Family-run destination with a strong local identity.",
    "Relaxed space for food, drink and community events.",
    "Small team offering carefully selected products.",
]
REVIEW_PHRASES = [
    "Friendly service and a welcoming atmosphere.",
    "Great value and easy to find from the town centre.",
    "The menu was varied and the staff were helpful.",
    "A pleasant visit, although the venue was busy.",
    "Would return and recommend it to friends.",
    "Good quality overall; accessibility information was useful.",
]

# Fixed ObjectIds keep review references valid when create_users.py runs.
USER_IDS = [
    "a0000000000000000000001",
    "a0000000000000000000002",
    "a0000000000000000000003",
    "a0000000000000000000004",
]


def object_id(value: str) -> dict[str, str]:
    return {"$oid": value}


def extended_date(value: datetime) -> dict[str, str]:
    return {"$date": value.isoformat().replace("+00:00", "Z")}


def business_object_id(index: int) -> str:
    return f"{0xB000000000000000000000 + index:024x}"


def business_name(index: int, category: str) -> str:
    prefix = NAME_PREFIXES[index % len(NAME_PREFIXES)]
    suffix = NAME_SUFFIXES[(index // len(NAME_PREFIXES)) % len(NAME_SUFFIXES)]
    return f"{prefix} {category} {suffix} {index + 1}"
```

```
 109 |
 110 |
 111 | def main() -> None:
 112 |     rng = random.Random(SEED)
 113 |     DATA_DIR.mkdir(parents=True, exist_ok=True)
 114 |
 115 |     businesses: list[dict] = []
 116 |     reviews: list[dict] = []
 117 |
 118 |     for business_index in range(BUSINESS_COUNT):
 119 |         town, town_lon, town_lat, postcode_prefix = TOWNS[
 120 |             business_index % len(TOWNS)
 121 |         ]
 122 |         categories = rng.sample(CATEGORIES, k=rng.choice([1, 1, 2, 2, 3]))
 123 |         longitude = town_lon + rng.uniform(-0.035, 0.035)
 124 |         latitude = town_lat + rng.uniform(-0.025, 0.025)
 125 |         created_at = BASE_DATE + timedelta(days=business_index * 3)
 126 |         business_oid = business_object_id(business_index)
 127 |
 128 |         ratings: list[int] = []
 129 |         for review_index in range(REVIEWS_PER_BUSINESS):
 130 |             rating = rng.randint(1, 5)
 131 |             ratings.append(rating)
 132 |             review_date = created_at + timedelta(
 133 |                 days=10 + review_index * 17,
 134 |                 hours=rng.randint(8, 20),
 135 |             )
 136 |             review_number = business_index * REVIEWS_PER_BUSINESS + review_index
 137 |             reviews.append(
 138 |                 {
 139 |                     "_id": object_id(f"{0xC0000000000000000000000 + review_number:024x}"),
 140 |                     "business_id": object_id(business_oid),
 141 |                     "user_id": object_id(USER_IDS[review_index % len(USER_IDS)]),
 142 |                     "rating": rating,
 143 |                     "text": rng.choice(REVIEW_PHRASES),
 144 |                     "created_at": extended_date(review_date),
 145 |                     "updated_at": extended_date(review_date),
 146 |                 }
 147 |             )
 148 |
 149 |         average_rating = round(sum(ratings) / len(ratings), 2)
 150 |         businesses.append(
 151 |             {
 152 |                 "_id": object_id(business_oid),
 153 |                 "name": business_name(business_index, categories[0]),
 154 |                 "town": town,
 155 |                 "address": f"{rng.randint(1, 180)} Sample Street",
 156 |                 "postcode": f"{postcode_prefix} {rng.randint(1, 9)}AA",
 157 |                 "categories": categories,
 158 |                 "description": rng.choice(DESCRIPTIONS),
 159 |                 "location": {
 160 |                     "type": "Point",
 161 |                     "coordinates": [round(longitude, 6), round(latitude, 6)],
 162 |                 },
 163 |                 "price_level": rng.randint(1, 4),
 164 |                 "active": True,
 165 |                 "rating": average_rating,
 166 |                 "review_count": len(ratings),
 167 |                 "created_at": extended_date(created_at),
 168 |                 "updated_at": extended_date(created_at),
 169 |             }
 170 |         )
 171 |
 172 |     (DATA_DIR / "businesses.json").write_text(
 173 |         json.dumps(businesses, indent=2, ensure_ascii=False), encoding="utf-8"
 174 |     )
 175 |     (DATA_DIR / "reviews.json").write_text(
 176 |         json.dumps(reviews, indent=2, ensure_ascii=False), encoding="utf-8"
 177 |     )
 178 |     print(
 179 |         f"Generated {len(businesses)} businesses and {len(reviews)} reviews in {DATA_DIR}"
 180 |     )
 181 |
 182 |
 183 | if __name__ == "__main__":
 184 |     main()
```

```
================================================================================================
FILE: biz_bp/requirements.txt
================================================================================================
   1 | Flask>=3.0,<4
   2 | flask-cors>=4.0,<7
   3 | PyJWT>=2.8,<3
```

```
   4 | bcrypt>=4.1,<6
   5 | pymongo>=4.6,<5

================================================================================================
FILE: biz_bp/.env.example
================================================================================================
   1 | MONGO_URI=mongodb://127.0.0.1:27017
   2 | MONGO_DB_NAME=locallens
   3 | JWT_SECRET=replace-with-a-long-random-value
   4 | JWT_EXPIRY_SECONDS=28800
   5 | HOST=127.0.0.1
   6 | PORT=5000
   7 | FLASK_DEBUG=0
   8 | ADMIN_PASSWORD=replace-with-a-local-demo-password
   9 | USER_PASSWORD=replace-with-a-local-demo-password
```